JAVA PROGRAMMING

Dr. R. Sahila Devi | Department of CSE | Rohini College of Engineering and Technology

Unit -2

Revision at a Glance

CONSTRUCTOR IN JAVA

A constructor is a special method in a class that is used to initialize objects.

KEY POINTS

- Name of the constructor is same as the class name.
- It has no return type, not even void.
- It is called automatically when an object is created.
- Used to initialize object attributes.

HOW CONSTRUCTOR WORKS

Class

Student

```
int rollNo;
String name;

Student()
{
   rollNo = 0;
   name = "Unknown";
}
```

new Student()

Constructor is called automatically

Object Created

s1 (Student Object)

rollNo = 0
name = "Unknown"

TYPES OF CONSTRUCTORS

1 Default Constructor (no-argument constructor)

```
Student()
{
   rollNo = 0;
   name = "Unknown";
}
```

Provided by compiler if no constructor is defined.

2 Parameterized Constructor

```
Student(int r, String n)
{
   rollNo = r;
   name = n;
}
```

Takes parameters to initialize objects.

EXAMPLE

```
class Student {
    int rollNo;
    String name;

    // Default Constructor
    Student() {
        rollNo = 0;
        name = "Unknown";
    }
    // Parameterized Constructor
    Student(int r, String n) {
        rollNo = r;
        name = n;
    }
}
public class Test {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student(101, "Alice");
    }
}
```

EXECUTION FLOW

1 Object Creation
Student s2 = new Student(101, "Alice");

2 Constructor Called
Student(int r, String n)

3 Initialization
rollNo = 101
name = "Alice"

4 Object Ready
s2 contains initialized values.

Constructor ensures that every object is properly initialized when it is created.

CONSTRUCTOR OVERLOADING IN JAVA

Constructor Overloading means having multiple constructors in a class with **different parameter lists**.

KEY POINTS

1. A class can have more than one constructor.
2. Constructors must differ in the number, type, or order of parameters.
3. Return type cannot be used to differentiate constructors.
4. It provides flexibility to initialize objects in different ways.
5. Called automatically when an object is created.

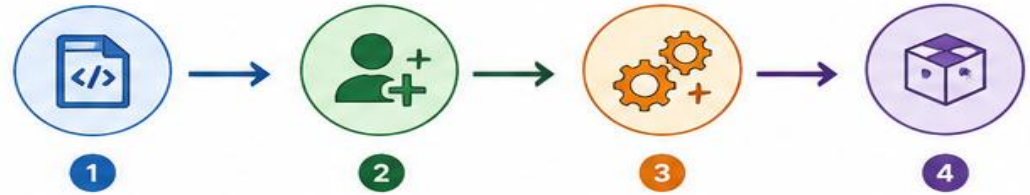
WHAT IS IT?

When a class has multiple constructors with different parameter lists, it is known as **Constructor Overloading**.

WHY USE IT?

- Increases code readability
- Provides multiple ways to create objects
- Reduces the need for setter methods
- Improves flexibility and reusability

HOW IT WORKS?



1. Class has multiple constructors with different parameters
2. When an object is created
3. JVM matches the arguments with the constructor signature
4. Matching constructor is called and object is initialized

EXAMPLE PROGRAM

```java
class Student {
    int rollNo;
    String name;
    int age;

    // 1. No-argument constructor
    Student() {
        rollNo = 0;
        name = "Unknown";
        age = 0;
    }

    // 2. Parameterized constructor (2 parameters)
    Student(int r, String n) {
        rollNo = r;
        name = n;
        age = 0;
    }

    // 3. Parameterized constructor (3 parameters)
    Student(int r, String n, int a) {
        rollNo = r;
        name = n;
        age = a;
    }
}

public class Test {
   public static void main(String[] args) {
       Student s1 = new Student();              // calls constructor 1
       Student s2 = new Student(101, "Alice");  // calls constructor 2
       Student s3 = new Student(102, "Bob", 20); // calls constructor 3
   }
}
```

CONSTRUCTORS IN THE ABOVE CLASS

Constructor	Signature	Description	Called When
1. No-argument	`Student()`	Initializes with default values	`new Student()`
2. Parameterized (2 params)	`Student(int r, String n)`	Initializes rollNo and name	`new Student(101, "Alice")`
3. Parameterized (3 params)	`Student(int r, String n, int a)`	Initializes rollNo, name and age	`new Student(102, "Bob", 20)`

EXAMPLES IN ACTION

`Student s1 = new Student();`

Constructor 1 Called

s1 Object	
rollNo	0
name	Unknown
age	0

`Student s2 = new Student(101, "Alice");`

Constructor 2 Called

s2 Object	
rollNo	101
name	Alice
age	0

`Student s3 = new Student(102, "Bob", 20);`

Constructor 3 Called

s3 Object	
rollNo	102
name	Bob
age	20

Constructor Overloading makes your program more flexible and easier to use.

METHOD OVERLOADING IN JAVA

Method Overloading allows a class to have more than one method with the **same name** but **different parameter lists** (number, type, or order of parameters).

KEY POINTS

- Methods must have the same name but different parameter lists.
- They must differ in number, type, or order of parameters.
- Return type alone cannot be used to differentiate methods.
- It improves code readability and flexibility.
- The appropriate method is decided by the compiler at compile time (Compile-time Polymorphism).

HOW IT WORKS?

1. Class contains multiple methods with the same name but different parameters.
2. When a method is called, the compiler looks at the arguments passed.
3. Compiler matches the arguments with the method signature.
4. The best matching method is selected and executed.

EXAMPLE – REAL LIFE ANALOGY

Think of a Camera:
It has the same function name "capture" but works differently based on the inputs you provide.

capture(...)	capture(int zoom)	capture(String mode, int quality)
(no parameter)	(with zoom level)	(mode and quality)
Default capture	Capture with zoom	Capture with mode & quality

EXAMPLE PROGRAM

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
    int add(int a, double b) {
        return (int)(a + b);
    }
}
public class Test {
  public static void main(String[] args) {
     Calculator c = new Calculator();
     System.out.println(c.add(10, 20));       // 30
     System.out.println(c.add(10.5, 20.5));   // 31.0
     System.out.println(c.add(10, 20, 30));   // 60
     System.out.println(c.add(10, 20.5));     // 30
  }
}
```

METHODS IN ABOVE CLASS (Calculator)

Method Name	Signature	Parameters	Return Type
add	add(int a, int b)	2 int	int
add	add(double a, double b)	2 double	double
add	add(int a, int b, int c)	3 int	int
add	add(int a, double b)	int, double	int

COMPILER DECISION (Compile-time Polymorphism)

Method to be executed is decided at compile time based on the arguments passed.

add(10, 20) →	add(int, int) →	Returns int
add(10.5, 20.5) →	add(double, double) →	Returns double
add(10, 20, 30) →	add(int, int, int) →	Returns int
add(10, 20.5) →	add(int, double) →	Returns int

RULES FOR METHOD OVERLOADING

- Same method name in the same class.
- Different parameter list.
- Can differ in number, type, or order of parameters.
- Return type alone is not sufficient.
- Access modifier can be same or different.

BENEFITS

- Improves readability
- Provides flexibility
- Easy to extend functionality

Method Overloading makes the program more intuitive, flexible and easier to maintain.

ACCESS SPECIFIERS IN JAVA

Access specifiers control the **visibility** of classes, variables, methods and constructors.

TYPES OF ACCESS SPECIFIERS

private	Accessible only within the same class.
default (package-private)	Accessible within the same package.
protected	Accessible within the same package and by subclasses (in other packages too).
public	Accessible from anywhere.

ACCESS LEVEL COMPARISON

Access Specifier	Same Class	Same Package	Subclass (Different Package)	Outside Package (Non-subclass)
private	✓	✗	✗	✗
default (package-private)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

REAL WORLD ANALOGY – OFFICE ACCESS

Access Specifier	Analogy	Explanation	Who Can Access?
private		Your personal drawer in the CEO's can access.	Only You
default (package-private)	Office room where your team works.	People in the same office (package) can access.	Team Members
protected		Manager and your team and acnagers (subclasses) even from other offices.	Team + Managers
public		Anyone from anywhere can access.	Everyone

EXAMPLE

```
class Demo {
    private int privateVar = 10;
    int defaultVar  = 20;           // default
    protected  int protectedVar   = 30;
    public int publicVar = 40;

    private void privateMethod() {
        System.out.println("private method");
    }
    void defaultMethod() {
        System.out.println("default method");
    protected void protectedMethod() {
        System.out.println("protected method");
    }
    public void publicMethod() {
        System.out.println("public method");
    }
}
```

WHERE CAN THEY BE USED?

- private → Fields, Methods, Constructors, Inner Classes
- default → Fields, Methods, Constructors, Inner Classes
- protected → Fields, Methods, Constructors, Inner Classes
- public → Fields, Methods, Constructors, Classes

KEY NOTES

- Top to bottom order of access: private < default < protected < public
- We should always restrict the access as much as possible.
- Public members are accessible everywhere, so use with care.
- Encapsulation is achieved by using private access.

SUMMARY

private (Only within the same class) → **default** (Within the same package) → **protected** (Same package + subclasses outside package) → **public** (Accessible from anywhere)

Better Security
Better Control
Better Code!

Remember!

Use the most restrictive access level that makes sense for your code.

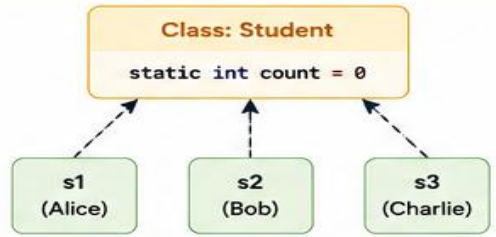
STATIC VARIABLES IN JAVA

A static variable (class variable) belongs to the class, not to any object. There is only one copy shared by all objects.

WHAT IS A STATIC VARIABLE?

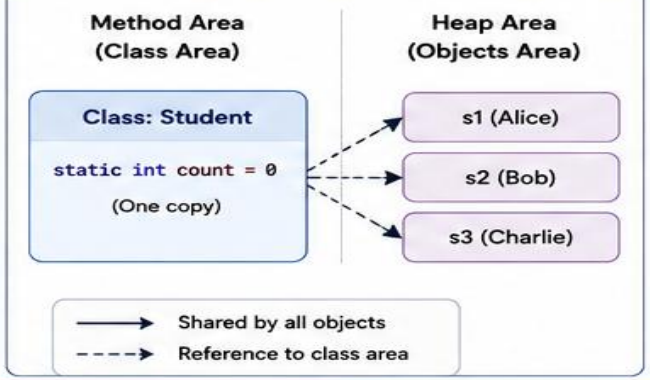
- Declared using the keyword `static`.
- There is only one copy of a static variable, no matter how many objects are created.
- Shared by all objects of the class.
- Memory is allocated when the class is loaded.
- Can be accessed using the class name.

HOW IT WORKS



All objects share the same static variable 'count'.
Any change through one object is reflected in all other objects.

MEMORY REPRESENTATION



KEY POINTS

- Belongs to the class, not to any object.
- Only one copy exists.
- Shared by all objects.
- Memory allocated when class is loaded.
- Accessed using class name (Preferred) or object reference.
- Changing it through one object changes it for all.

EXAMPLE PROGRAM

```
class Student {
     static int count = 0;     // static variable
     String name;

     Student(String name) {
        this.name = name;
        count++;      // increment count for every object
     }

     void display() {
         System.out.println(name + " | Total Students: " + count);
     }
}

public class TestStatic {
   public static void main(String[] args) {
      Student s1 = new Student("Alice");
      Student s2 = new Student("Bob");
      Student s3 = new Student("Charlie");

      s1.display();
      s2.display();
      s3.display();
   }
}
```

Explanation

- When each object is created, the constructor increases the static variable count.
- All objects share the same count.

OUTPUT

```
Alice   | Total Students:  1
Bob     | Total Students:  2
Charlie | Total Students:  3
```

ACCESSING STATIC VARIABLE

Using Class Name (Recommended)	Using Object Reference (Allowed but not preferred)
`int total = Student.count;` ✓	`int total = s1.count;` ⚠

Prefer accessing static variables using class name for better readability and maintainability.

INSTANCE vs STATIC VARIABLES

Feature	Instance Variable	Static Variable
Belongs to	Object	Class
Copies	Each object has its own copy	Only one copy for all objects
Memory	Allocated when object is created	Allocated when class is loaded
Access	Through object reference	Through class name (preferred)
Can we use `this` / `super` ?	Yes	No
Example	name, age	count, totalMarks

COMMON USES

- 001 Counters (e.g., total number of objects)
- Configuration Settings
- Utility Data shared by all objects
- Logging information

Remember: Static variable is shared by all. Change it once, it changes everywhere!

STATIC METHODS IN JAVA

A static method belongs to the class, not to any object. It can be called using the class name.

WHAT IS A STATIC METHOD?

- Declared using the keyword **static**.
- Belongs to the class, not to any object.
- Can be called using class name without creating an object.
- Can access only **static** data directly.
- Cannot use **this** or **super** keyword.

Why Static Method?
To perform common operations that are related to the class as a whole (Utility methods).

HOW IT WORKS

Class: MathUtil
static method
add(int a, int b)

↓

Call using Class Name
MathUtil.add(10, 20)

↓

Method is executed
(No object required)

CALLING STATIC METHOD

1. Using Class Name (Recommended)

```
int sum = MathUtil.add(10, 20);
```

No object is needed.

2. Using Object Reference (Allowed but not preferred)

```
MathUtil obj = new MathUtil();
int sum = obj.add(10, 20);
```

Works, but not recommended.

FEATURES

- Belongs to the class.
- No need to create an object.
- Called using class name.
- Can access only static data directly.
- Cannot use this or super keyword.
- Useful for utility or helper methods.

EXAMPLE PROGRAM

```
class Calculator {
    static int count = 0;   // static variable

    static int add(int a, int b) {   // static method
        count++;
        return a + b;
    }

    static void showCount() {
        System.out.println("Total calls to add(): " + count);
    }
}

public class TestStaticMethod {
    public static void main(String[] args) {
        int result1 = Calculator.add(10, 20);  // using class name
        int result2 = Calculator.add(5, 15);   // using class name

        System.out.println("Result1: " + result1);
        System.out.println("Result2: " + result2);

        Calculator.showCount();              // static method
    }
}
```

OUTPUT

```
Result1: 30
Result2: 20
Total calls to add(): 2
```

ACCESSING FROM ANOTHER CLASS

```
class UtilDemo {
    static void greet() {
        System.out.println("Hello from static method!");
    }
}

class AnotherClass {
    public static void main(String[] args) {
        UtilDemo.greet();    // call static method
    }
}
```

STATIC METHOD vs INSTANCE METHOD

Feature	Static Method	Instance Method
Belongs to	Class	Object
Need object to call	No	Yes
Call using	Class name	Object reference
Access to instance data	No (only static data)	Yes (static + instance data)
this / super keyword	Cannot be used	Can be used
Memory	Loaded once (shared)	Created with each object
Example	`Math.sqrt(), System.gc()`	`toString(), getName()`

IMPORTANT NOTES

- A static method can call another static method or access static variables directly.
- It cannot call a non-static (instance) method or access instance variables directly.
- If you need to access instance members, create an object.

Example: main() method in Java is static, so that JVM can call it without creating any object.

COMMON USES OF STATIC METHODS

- Utility functions (e.g., Math class)
- Date / Time operations (e.g., LocalDate.now())
- Validation methods
- Helper methods
- Printing / Logging

BEST PRACTICE

- Use static methods for operations that are common and independent of object state.
- Avoid making everything static. Use only when required.

Remember: Static methods are class level methods – Use class name, save memory and write better code!

Real Coding Demonstrations

Hands-on Java examples: variables, classes & objects, loops and arrays

DEMO 2

Constructors in Java

```java
class Student {
    int id;
    String name;
    double cgpa;

    // Default Constructor
    Student() {
        id = 0;
        name = "Unknown";
        cgpa = 0.0;
    }
    // Parameterized Constructor
    Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }

    // Display Method
    void show() {
        System.out.println("ID:-" + id);
        System.out.println("Name:-" + name);
        System.out.println("CGPA:-" + cgpa);
    }
}
public class ConstructorDemo {
    public static void main(String[] args) {

        // Using default constructor
        Student s1 = new Student();
        s1.show();
        System.out.println();

        // Using parameterized constructor
        Student s2 = new Student(101, "Alice", 9.25);
        s2.show();
    }
}
```

CONSOLE OUTPUT

```
ID: 0
Name: Unknown
CGPA: 0.0

ID: 101
Name: Alice
CGPA: 9.25
```

KEY TAKEAWAYS

- A constructor is a special method used to initialize objects.
- It has the same name as the class.
- It does not have a return type (not even void).
- It is called automatically when an object is created.
- Constructors can be:
  - Default Constructor (no parameters)
  - Parameterized Constructor (with parameters)

DEMO 2

Constructor Overloading in Java

```java
class Student {
    int id;
    String name;
    double cgpa;

    // 1. Default Constructor
    Student() {
        id = 0;
        name = "Unknown";
        cgpa = 0.0;
    }
    // 2. Parameterized Constructor (2 parameters)
    Student(int id, String name) {
        this.id = id;
        this.name = name;
        this.cgpa = 0.0;
    }
    // 3. Parameterized Constructor (3 parameters)
    Student(int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }
    // Display method
    void show() {
        System.out.println("ID: " + id);
        System.out.println("Name: " + name);
        System.out.println("CGPA: " + cgpa);
        System.out.println("----------------");
    }
}
public class ConstructorOverloadingDemo {
    public static void main(String[] args) {
        Student s1 = new Student();                    // default constructor
        Student s2 = new Student(101, "Alice");        // 2-parameterized constructor
        Student s3 = new Student(102, "Bob", 8.75); // 3-parameter constructor

        s1.show();
        s2.show();
        s3.show();
    }
}
```

CONSOLE OUTPUT

```
ID: 0
Name: Unknown
CGPA: 0.0
----------------

ID: 101
Name: Alice
CGPA: 0.0
----------------

ID: 102
Name: Bob
CGPA: 8.75
----------------
```

KEY TAKEAWAYS

- Constructor overloading means having multiple constructors with different parameter lists in the same class.
- It allows us to create objects in different ways.
- The compiler chooses the appropriate constructor based on the arguments provided.
- Types of constructors used here:
  1. Default Constructor (no parameters)
  2. Parameterized Constructor (2 parameters)
  3. Parameterized Constructor (3 parameters)

DEMO 3

Method Overloading in Java

Method overloading allows a class to have more than one method with the same name, but with different parameter lists (type, number or order of parameters).

```java
class Calculator {
    // 1. Method with two int parameters
    int add(int a, int b) {
        return a + b;
    }
    // 2. Method with three int parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }
    // 3. Method with two double parameters
    double add(double a, double b) {
        return a + b;
    }
    // 4. Method with one int parameter
    int add(int a) {
        return a;
    }
}
public class MethodOverloadingDemo {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        // Calling method with two int arguments
        System.out.println("Add(10, 20) = " + calc.add(10, 20));

        // Calling method with three int arguments
        System.out.println("Add(10, 20, 30) = " + calc.add(10, 20, 30));

        // Calling method with two double arguments
        System.out.println("Add(10.5, 20.5) = " + calc.add(10.5, 20.5));

        // Calling method with one int argument
        System.out.println("Add(50) = " + calc.add(50));
    }
}
```

CONSOLE OUTPUT

```
Add(10, 20) = 30
Add(10, 20, 30) = 60
Add(10.5, 20.5) = 31.0
Add(50) = 50
```

KEY TAKEAWAYS

- Method overloading means having multiple methods with the same name in the same class.
- The methods must have different parameter lists:
  - Different number of parameters
  - Different type of parameters
  - Different order of parameters
- Return type may be same or different.
- It improves code readability and reusability.

Note:
Return type alone cannot be used to overload a method.

DEMO 4

Static Variables and Methods in Java

Static members belong to the class, not to any object. They can be accessed using the class name without creating an object.

```java
class Counter {
    // Static variable (shared by all objects)
    static int count = 0;

    // Static method
    static void displayCount() {
        System.out.println("Total count: " + count);
    }

    // Instance method
    void increment() {
        count++;  // modifies the static variable
        System.out.println("Count incremented. Current count: " + count);
    }
}

public class StaticDemo {
    public static void main(String[] args) {
        // Access static variable and method using class name
        System.out.println("Initial count: " + Counter.count);
        Counter.displayCount();

        // Creating objects
        Counter c1 = new Counter();
        Counter c2 = new Counter();

        // Calling instance method (changes static variable)
        c1.increment();
        c2.increment();
        c1.increment();

        // Access again using class name
        Counter.displayCount();
    }
}
```

CONSOLE OUTPUT

```
Initial count: 0
Total count: 0
Count incremented. Current count: 1
Count incremented. Current count: 2
Count incremented. Current count: 3
Total count: 3
```

KEY TAKEAWAYS

- Static variable is shared by all objects of the class.
- It is allocated memory when the class is loaded.
- Static method can be called using the class name.
- Static method can access only static data directly.
- Static members exist even if no object is created.
- Used for common data, utility functions, counters, configuration, etc.

IMPORTANT NOTE

Static methods cannot use `this` or `super` keyword because they do not belong to any object.

```java
static void show() {
    System.out.println(this); // Error
}
```

STATIC vs INSTANCE

Feature	Static (class level)	Instance (object level)
Belongs to	Class	Object
Created	When class is loaded	When object is created
Accessed using	Class name	Object reference
Memory	One copy (shared)	Separate copy for each object
Can exist without object?	Yes	No
Access in static method	Only static members	Both static & instance members

Knowledge Check

Five quick questions to test what we've covered so far

TOPIC 1: CONSTRUCTORS

A constructor in Java is a special method that is used to initialize an object. It is called automatically when an object is created.

CLASS

Student

- int id;
- String name;
- Student() { }
- Student(int id, String name) { }

new Student()

OBJECT

s1

id = 101
name = Alice

Constructor called automatically

1 **What is the primary purpose of a constructor in Java?**

A) Perform calculations
B) Initialize objects
C) Delete objects
D) Display output

Answer:
B) Initialize objects

2 **A constructor has the same name as the:**

A) Method
B) Object
C) Class
D) Package

Answer:
C) Class

Class Student
Student() { }

3 **Which of the following is true about constructors?**

A) They have a return type.
B) They are called manually.
C) They are called automatically when an object is created.
D) They must be static.

Answer:
C) They are called automatically when an object is created.

4 **Which keyword is used to create an object?**

A) class
B) create
C) new
D) object

Answer:
C) new

5 **Which constructor takes arguments?**

A) Default Constructor
B) Parameterized Constructor
C) Static Constructor
D) Final Constructor

Answer:
B) Parameterized Constructor

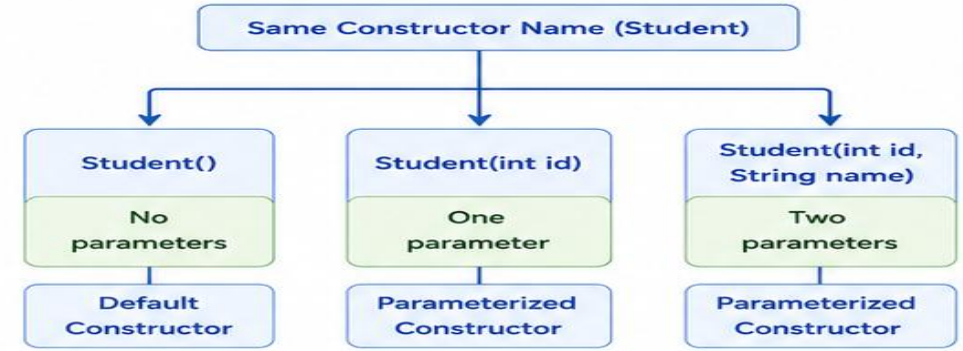
```
Student(int id, String name) {
    this.id = id;
    this.name = name;
}
```

TOPIC 2: CONSTRUCTOR OVERLOADING

Constructor overloading allows a class to have multiple constructors with the same name but different parameter lists.

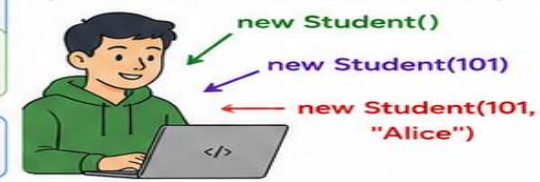
CLASS: Student

```
class Student {
  int id;
  String name;

Student() { }
Student(int id) { }
Student(int id, String name) { }
}
```



How it works?

When an object is created, the compiler chooses the appropriate constructor based on the arguments provided.



1 **Constructor overloading means:**

A) Constructors with different names
B) Multiple constructors with different parameter lists
C) Multiple classes
D) Multiple objects

✔ Answer: B) Multiple constructors with different parameter lists

2 **Constructor overloading is achieved by changing:**

A) Constructor name
B) Return type
C) Parameter list
D) Access modifier only

✔ Answer: C) Parameter list

3 **Which of the following is NOT valid for constructor overloading?**

A) Different number of parameters
B) Different parameter types
C) Different parameter order
D) Different return type only

✔ Answer: D) Different return type only

4 **Who decides which constructor is called?**

A) JVM
B) Compiler
C) User
D) Object

✔ Answer: B) Compiler

5 **Constructor overloading provides:**

A) Security
B) Flexibility
C) Inheritance
D) Encapsulation

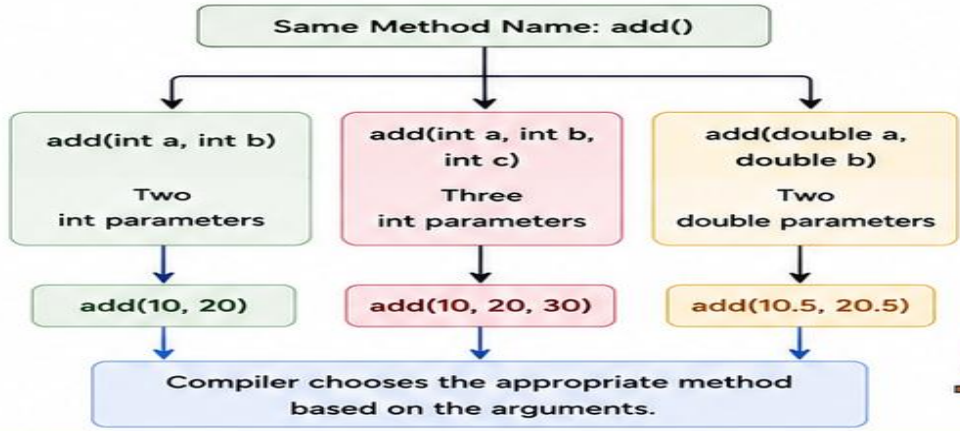
✔ Answer: B) Flexibility

TOPIC 3: METHOD OVERLOADING

Method overloading allows a class to have more than one method with the same name but different parameter lists.

EXAMPLE CLASS

```
class Calculator {
  int add(int a, int b) {
    return a + b;
  }
  int add(int a, int b, int c) {
    return a + b + c;
  }
  double add(double a, double b) {
    return a + b;
  }
}
```



HOW IT WORKS

When a method is called, the compiler matches the method name and parameter list, then executes the best matching method.

1 **Method overloading means:**

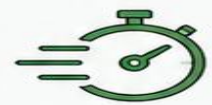
A) Same method name with same parameters
B) Same method name with different parameter lists
C) Different method names
D) Same return type only

Answer: B) Same method name with different parameter lists

2 **Method overloading is an example of:**

A) Runtime Polymorphism
B) Compile-time Polymorphism
C) Inheritance
D) Abstraction

Answer: B) Compile-time Polymorphism



3 **Which of the following can be overloaded?**

A) Constructors
B) Methods
C) Both A and B
D) Variables

Answer: C) Both A and B

4 **Which of the following cannot distinguish overloaded methods?**

A) Number of parameters
B) Parameter type
C) Parameter order
D) Return type alone

Answer: D) Return type alone

5 **Which method will be called?**

```
add(10, 20);
```

A) add(double, double)
B) add(int, int)
C) add(int, int, int)
D) None

Answer: B) add(int, int)

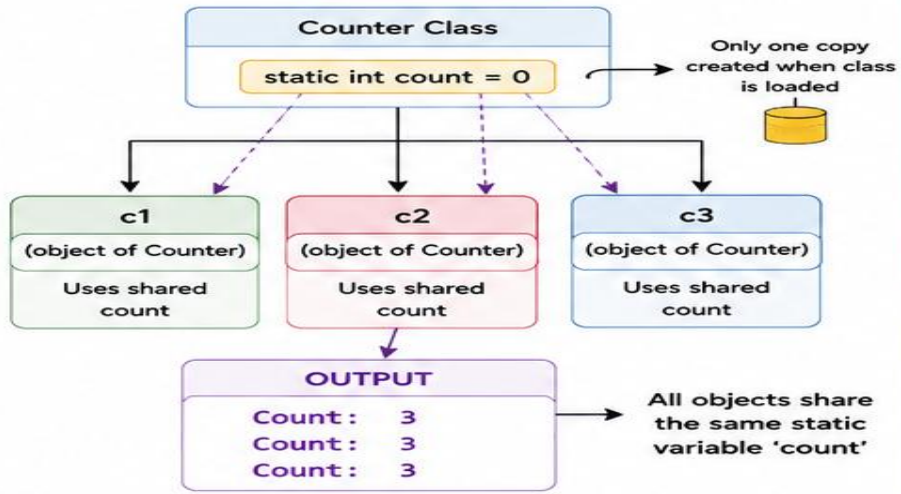
TOPIC 4: STATIC VARIABLE

A **static variable** (class variable) belongs to the class, not to any object. It is **shared by all objects** of the class. There is only **one copy** of a static variable.

EXAMPLE

```
class Counter {
   static int count = 0;

   Counter() {
      count++;    // shared by all
   }

   void display() {
      System.out.println("Count: " + count);
   }
}

public class StaticDemo {
   public static void main(String[] args) {
      Counter c1 = new Counter();
      Counter c2 = new Counter();
      Counter c3 = new Counter();

      c1.display(); // 3
      c2.display(); // 3
      c3.display(); // 3
   }
}
```

HOW IT WORKS



KEY POINTS

- Declared using 'static' keyword.
- Belongs to the class.
- One copy is shared by all objects.
- Memory is allocated when the class is loaded.
- Can be accessed using class name.
- Commonly used for counters, configuration, etc.

ACCESSING STATIC VARIABLE

Best Practice:

`Counter.count`

Also valid (but not recommended):

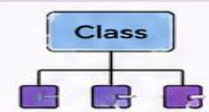
`c1.count`

---

1 A static variable belongs to:

A) Object B) Method C) Class D) Package

Answer: C) Class



---

2 How many copies of a static variable exist?

A) One per object B) One per class C) Two D) Unlimited

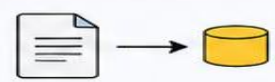
Answer: B) One per class

---

3 Static variables are created when:

A) Object is created B) Program ends C) Class is loaded D) Method is called

Answer: C) Class is loaded

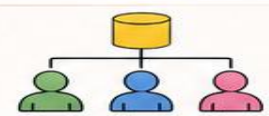


---

4 Static variables are shared by:

A) One object B) All objects C) One method D) None

Answer: B) All objects



---

5 Preferred way to access a static variable is:

A) Object name B) Class name C) Package name D) Constructor

Answer: B) Class name

TOPIC 5: STATIC METHOD

A **static method** belongs to the **class** rather than any **object**.
It can be called without creating an object of the class.
It can access only **static data** directly.

EXAMPLE

```
class MathUtil {
    // static method
    public static int add(int a, int b) {
        return a + b;
    }

    public static double pi() {
        return 3.14159;
    }
}

public class StaticMethodDemo {
  public static void main(String[] args) {
      int sum = MathUtil.add(10, 20);
      double p = MathUtil.pi();
      System.out.println("Sum = " + sum);
      System.out.println("Pi = " + p);
  }
}
```

HOW IT WORKS

MathUtil Class

```
static int add(int a, int b)
static double pi()
```

→ Belongs to the class

Call using Class Name

```
MathUtil.add(10, 20)
MathUtil.pi()
```

→ No object needed

Output

```
Sum = 30
Pi  = 3.14159
```

→ Method executes

KEY POINTS

- Declared using 'static' keyword.
- Belongs to the class.
- Can be called using class name.
- Can be called without creating an object.
- Can access only static variables and other static methods directly.
- Cannot use 'this' or 'super' keyword.
- Commonly used for utility functions.

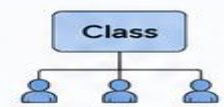
SYNTAX

```
access_modifier static return_type
 methodName(parameters) {
    // body
}
```

1 **Static methods belong to the:**

A) Object B) Class C) Constructor D) Interface

✓ Answer: B) Class



2 **Can a static method be called without creating an object?**

A) Yes B) No C) Sometimes D) Only with inheritance

✓ Answer: A) Yes

3 **Which keyword declares a static method?**

A) final B) public C) static D) void

✓ Answer: C) static

```
static void show() {
    // body
}
```

4 **A static method can directly access:**

A) Instance variables only B) Static variables only C) Local variables only D) Constructors only

✓ Answer: B) Static variables only

5 **Which keyword cannot be used inside a static method?**

A) new B) class C) this D) static

✓ Answer: C) this

Q & A

Questions, thoughts, or something you'd like demoed again?

Thank You

Dr. R. Sahila Devi

Department of CSE · Rohini College of Engineering and Technology